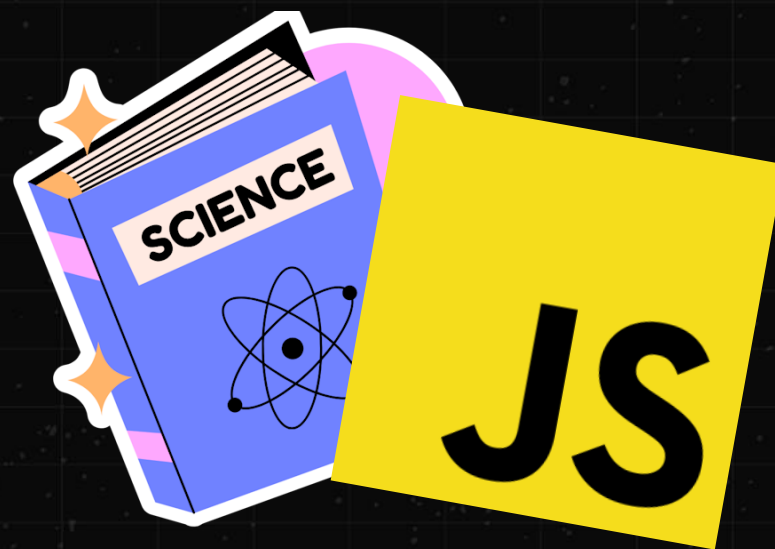


ЛЕКЦИЯ #7

CS BO FRONTEND



ЧТО СЕГОДНЯ НА ПОВЕСТКЕ

- * Продолжаем говорить про массивы: разбираем типизированные массивы JS
- * Познакомимся с концепцией **объектов представления данных**
- * Выравниванием памяти и порядком следования байт
- * И разберем такие фундаментальные структуры данных как **кортеж, структура и перечисление (размеченное объединение)**

МЫ УЖЕ ЗНАКОМЫ С МАССИВАМИ

- * Это фиксированная непрерывная область памяти представляющая **последовательный список элементов одного размера**
- * Массивы в JavaScript представляются куда **более могущественной структурой**
- * Они **поддерживает элементы разных типов (гетерогенность) и расширяемость**
- * А также могут содержать «дырки»
- * Но в JS есть и **другие виды массивов**,
которые более подходят под классическое описание

ВСТРЕЧАЙТЕ, ТИПИЗИРОВАННЫЕ МАССИВЫ

```
const arr1 = new Float32Array([1.23, 4.2, 0]);  
  
// 1.2300000190734863 4.199999809265137 0  
console.log(arr1[0], arr1[1], arr1[2]);  
  
const arr2 = new BigInt64Array([10n, 5n, 4n]);  
  
// 10n 5n 4n  
console.log(arr2[0], arr2[1], arr2[2]);  
  
const arr3 = new Int32Array(500);  
  
// 0 0 0 500  
console.log(arr3[0], arr3[1], arr3[2], arr3.length);
```

ОНИ ГОРАЗДО ПРОЩЕ **Array**

- * Никаких дырок – вся память сразу размечается нулями
- * Никакой гетерогенности – можем хранить только элементы конкретного типа
- * Никаких ссылок – элементы всегда хранят своё значение в самом массиве
- * API похож на **Array**, но отличается в нюансах

В ЧЕМ СМЫСЛ
ТАКИХ МАССИВОВ?



КОНТРОЛЬ И ПЕРЕНОСИМОСТЬ

- * За счет богатого набора числовых типов мы можем **точно контролировать расход памяти**
- * А строгий контракт на хранение элементов позволяет **лучше оптимизировать работу кэша, JIT и сборщика мусора**
- * Простота реализации позволяет легко передавать такие массивы **в другие потоки/процессы или по сети без сериализации**
- * Например, `Array` обычно сериализуются в JSON строку

НО ЭТО РАБОТАЕТ И В ОБРАТНУЮ СТОРОНУ

- * Мы можем прочитать любой бинарный файл как типизированный массив
- * Это открывает возможности для работы на стороне JS со сторонними бинарными форматами, например, [PDF](#) или [PSD](#)
- * Сделать [полифил для нового формата изображений](#), которые еще не поддерживаются нативно
- * Или даже работать с бинарными данными из другого ЯП

КАКИЕ **ТИПЫ** ПОДДЕРЖИВАЮТСЯ?



ВСЕ НЕОБХОДИМОЕ

- * Целые знаковые числа: `Int8Array`, `Int16Array`, `Int32Array`, `BigInt64Array`
- * Целые беззнаковые числа: `Uint8Array` и `Uint8ClampedArray`,
`Uint16Array`, `Uint32Array`, `BigUint64Array`
- * Числа с плавающей точкой: `Float16Array`, `Float32Array`, `Float64Array`

ТУТ СРАЗУ НАДО ПОНЯТЬ

- * В JS всего 2 числовых типа: **number** и **bigint**
- * Если мы читаем некоторый элемент массива, то он будет автоматически преобразован в один из двух типов (в **bigint** из **BigInt64Array/BigUint64Array**)

```
const arr = new Float32Array([1.23, 4.2, 0]);

// float32 будет преобразован в number, который мы уже прочитаем
console.log(arr[0]);

// number будет преобразован в float32,
// который уже будет храниться в памяти массива
arr[0] = 2.3;
```

ПЕРЕПОЛНЕНИЯ И ПОТЕРЯ ТОЧНОСТИ



```
const arr1 = new Uint8Array(10);

// В Uint8 255 максимальное целое,
// поэтому 256 переполниться и станет 0
arr1[0] = 256;
console.log(arr1[0]); // 0

// Знаковый бит в Uint8 считается за обычное значение,
// поэтому -1 будет интерпретироваться как 255
arr1[1] = -1;
console.log(arr1[1]); // 255
```



```
const arr2 = new Uint8ClampedArray(10);

// Uint8ClampedArray особым образом
// обрабатывает переполнения
// вписывая число в границы
arr2[0] = 256;
console.log(arr2[0]); // 255

arr2[1] = -1;
console.log(arr2[1]); // 0
```



```
const arr3 = new Float16Array(10);

// В number мантисса состоит из 52 бит,
// а в float16 из 10 бит,
// поэтому точность представления числа
// становится меньше
arr3[0] = 1.3;
console.log(arr3[0]); // 1.2998046875
```

second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

СТОЯНА, СТОЯНААА!

ПОНИМАЮ, ЧТО ЭТО МОЖЕТ СБИВАТЬ

- * Держите в голове, что у массива есть **внешний и внутренний интерфейс**
- * Внутренний отвечает за то, **как массив хранит свои данные в памяти**
- * Внешний за **«общение» с остальными частями программы на JS**
- * Когда вы сохраняете внешние данные в массив, **они преобразуются в тот формат, в котором будут храниться**
- * Когда читаете – **преобразуются «обратно»** в один из встроенных типов JS

СОЗДАНИЕ МАССИВА

- * В конструктор массива можно передать число, которое **будет задавать длину создаваемого массива**
- * Типизированный массив **сразу инициализирует все элементы** без дырок
- * Еще в конструктор можно передать **обычный или другой типизированный массив**
- * А с помощью **.of** создать массив на основе любого Iterable-объекта

```
const set = new Set([1, 2, 3]);  
console.log(Int32Array.of(...set));
```

СОЗДАНИЕ ИЗ БАЙТОВОГО БУФЕРА

- * Любой типизированный массив можно создать **на основе «сырой памяти»**
- * В JS для представления таких значений есть `ArrayBuffer` и `SharedArrayBuffer`
- * Про `SharedArrayBuffer` поговорим в отдельной лекции
- * Главная особенность такого создания, что типизированный массив **не будет аллоцировать новую память**, а использовать переданную
- * Фактически, в таком случае **просто создается «обертка»**
- * Если в сырой памяти есть какие-то значения, **то они будут интерпретироваться как значения массива**



```
const buffer = new ArrayBuffer(1024); // Размер буфера указывается в байтах

const arr1 = new Float64Array(buffer);

console.log(arr1.length); // 128, так как один элемент Float64 занимает 0 байт
console.log(arr1[0]); // 0, так как вся память проинициализирована нулями по умолчанию

// Можно инициализировать небольшой буфер с возможностью роста
const buffer2 = new ArrayBuffer(8, {maxByteLength: 1024});

const arr2 = new BigInt64Array(buffer2);

console.log(arr2.length); // 1
console.log(arr2[0]); // 0n

// Меняем размер буфера
arr2.buffer.resize(32);

console.log(arr2.length); // 4
```

РЕФЛЕКСИРУЕМ

- * Мы можем заранее аллоцировать память под типизированный массив
- * А также заложить возможность её роста в обозначенном диапазоне
- * Можно получить доступ к буферу типизированного массива через геттер `.buffer`



```
// Можно прочитать любой ответ из сети, как ArrayBuffer
const myFile = new Uint8Array(await (await fetch("http://my-site.com")).arrayBuffer());

// Или передать по сети
fetch("http://my-site.com", {body: myFile})

const reader = new FileReader();

reader.onload = (e) => {
  const anotherFile = new Uint8Array(e.target?.result);
};

// Или прочитать файл с файловой системы
reader.readAsArrayBuffer(fileSource);
```

РЕФЛЕКСИРУЕМ

- * `ArrayBuffer` это честное представление потока байт
- * Мы можем прочитать любые данные таким образом



```
const buffer = new ArrayBuffer(1024); // Размер буфера указывается в байтах

console.log(buffer.byteLength); // 1024
console.log(buffer.resizable); // false

// Можно передать владение буфера в другую память или поток
console.log(buffer.detached); // false

// Создает копию памяти и новый ArrayBuffer с возможностью задать диапазон копирования
console.log(buffer.slice(0, 10));

// Отдали владение буфером и явно указали, что нас интересуют лишь первые 64 байта.
// Допускается задать больше размер, чем у исходного буфера (но не больше maxByteLength если он задан).
// Еще есть метод transferToFixedLength, который убирает возможно resizable у буфера, если таковая имелась.
const newBuffer = buffer.transfer(64);

console.log(newBuffer.byteLength); // 64

console.log(buffer.detached); // true
console.log(buffer.byteLength); // 0, так как мы отдали владения буфером
```

РЕФЛЕКСИРУЕМ

- * `ArrayBuffer` позволяет не только скопировать свои данные, но и **передать владение ими**
- * При этом изначальный владелец **теряет доступ к такой памяти**
- * Это позволяет **передавать память в другой поток без копирования**
- * А `SharedArrayBuffer` может одновременно **использоваться из разных потоков**
- * Обычные массивы **всегда копируются полностью**



```
const arr1 = new Uint8Array([34, 45, 10, 75]);

// Передаем не сам массив, а его байтовый буфер
const arr2 = new Float32Array(arr1.buffer);

console.log(arr2[0]); // 9055522
console.log([...arr1]); // По-прежнему [34, 45, 10, 75]

arr1[0] = 100;

console.log(arr2[0]); // Число изменилось на 9055588
console.log([...arr1]); // [100, 45, 10, 75]

arr2[0] = -45.33444;

console.log(arr2[0]); // -45.33443832397461
console.log([...arr1]); // Теперь байты такие [119, 86, 53, 194]
```

РЕФЛЕКСИРУЕМ

- * При создании типизированного массива из буфера у одной памяти может быть больше одного потребителя
- * При этом каждый потребитель интерпретирует память согласно своим контрактам
- * Это особенно важно при записи в массив

second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

СТОЯНА, СТОЯНААА!

ВЫДЕЛЯЕМ ГЛАВНОЕ

- * Все в компьютере – **это просто поток байт**
- * `ArrayBuffer` – это буквально и есть поток байт фиксированного размера
- * Самостоятельно работать с этим потоком мы не можем,
но **можем использовать его для создания типизированного массива**
- * Типизированный массив просто добавляет контракты на интерпретацию байт –
память при этом не копируется

VIEW НАД ДАННЫМИ

- * Подход, когда у нас есть **отдельно данные и представление (view)** для этих данных очень популярен и удобен в самых разных задачах
- * Главный тут плюс в том, что мы можем иметь много потребителей одних и тех же данных, но **предоставлять им разный API доступа**
- * При этом **копировать данные для каждого потребителя** не обязательно

```
console.log(ArrayBuffer.isView(new ArrayBuffer(1024))); // false  
console.log(ArrayBuffer.isView(new Uint8Array(10)));    // true
```

```
class User {
  get has18() {
    return new Date().getFullYear() - this.#data.birthdate.getFullYear() >= 18;
  }

  get canProgramming() {
    return this.#data.skills.includes("programming");
  }

  get name() {
    return this.#data.name;
  }

  set name(name) {
    this.#data.name = name;
  }

  #data;

  constructor(data) {
    this.#data = data;
  }
}

class Users {
  #data;

  constructor(data) {
    this.#data = data;
  }

  at(index) {
    return new User(this.#data.at(index));
  }
}
```

```
const data = [
  {name: "Bob", birthdate: new Date(1990, 1, 1), skills: ["programming", "sing"]},
  {name: "Ben", birthdate: new Date(1989, 11, 10), skills: ["sing"]},
];

const users = new Users(data);

console.log(users.at(0).has18);
```

РЕФЛЕКСИРУЕМ

- * Мы создали абстракцию над «сырыми» данными: часть данных сокрыли, добавили новые аксессоры или методы
- * Избегаем избыточного копирования – можно добавить копирование при записи, если это потребуется
- * Для разных потребителей мы можем делать разные представления
- * Особенно это удобно для различных миграций

КРУТО, НО **ЗАЧЕМ** НАМ VIEW
ДЛЯ ТИПИЗИРОВАННОГО
МАССИВА?



МАСШТАБИРУЕМ ИДЕЮ

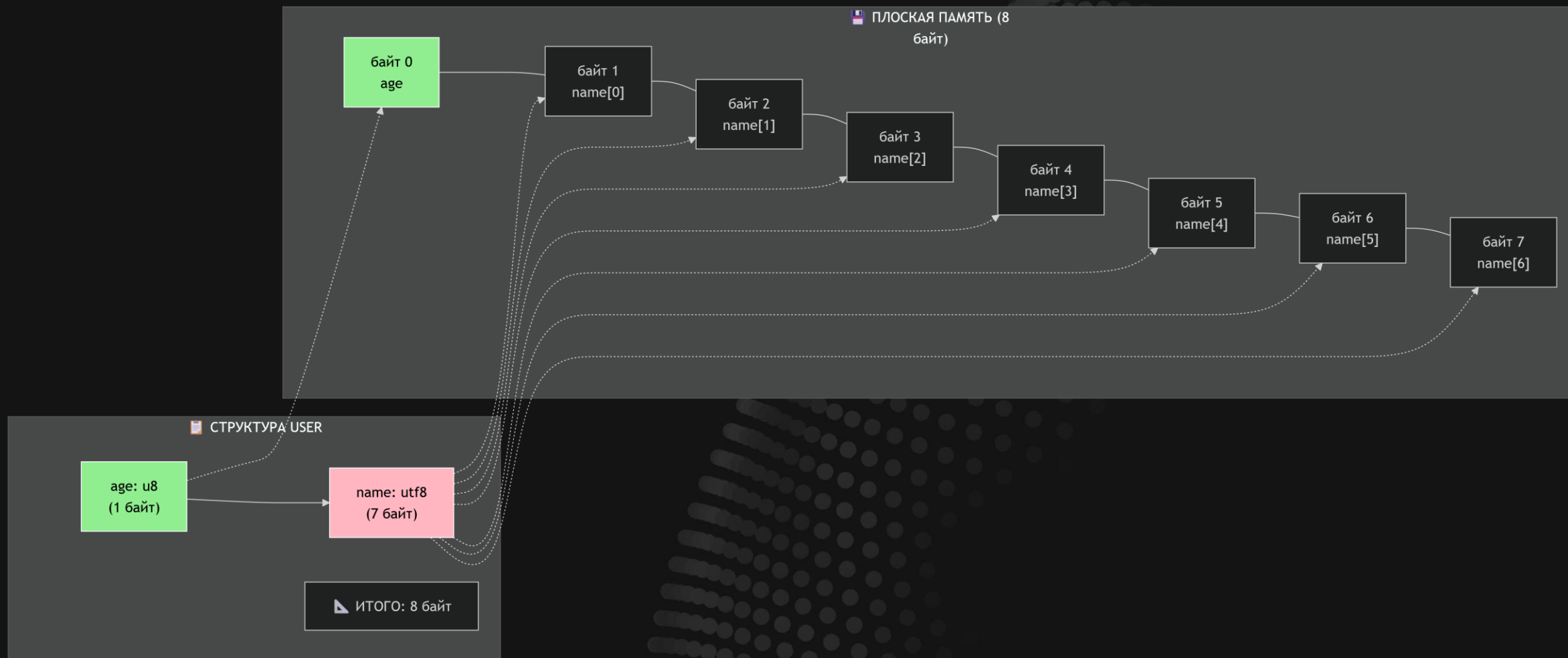
- * `ArrayBuffer` представляет собой «уменьшенный» аналог всей памяти на компьютере
- * В разных его частях мы можем **хранить данные разных типов**
- * Более того, можем сделать структуру, где **разные поля смогут иметь разный типы**

ОПИШЕМ ТИП ДАННЫХ USER

```
const U8 = {  
  bytes: 1,  
  view: Uint8Array  
};  
  
const Utf8_7 = {  
  bytes: 7,  
  view: Uint8Array  
};  
  
const User = {  
  age: U8, // Для возраста пользователя хватит 1 байта  
  name: Utf8_7 // На имя в кодировке Utf8 отвели 7 байт  
};
```

МЫ СОЗДАЛИ СХЕМУ ДАННЫХ

- ✱ Используя эту схему мы можем как закодировать, так и декодировать данные
- ✱ Закодированные данные **лежат рядом в памяти**, что благоприятно скажется на кэшировании
- ✱ Общий размер всей структуры **равен сумме размеров всех полей** – в нашем случае это 64 бита



ЗНАКОМЬТЕСЬ, СТРУКТУРА

- * Еще одна **важнейшая фундаментальная структура данных** в программировании
- * В отличие от массива, у нас появляется возможность **хранить в памяти элементы с разным размером или типом**
- * Размер структуры известен заранее, поэтому **можно сделать массив таких структур**
- * Как и сделать **другую структуру/массив полем структуры**
- * Данные в памяти будут лежать «рядом»

НА ПРИМЕРЕ RUST

```
fn main() {  
    struct User {  
        age: u8,  
        name: String  
    }  
  
    // У нас на руках адрес первого байта структуры,  
    // поэтому доступ к name можно высчитать статически  
    let bob = User { age: 20, name: "Bob".to_string() };  
    println!("Hello, {}!", bob.name);  
}
```

ЭТО КАК ОБЪЕКТЫ В JAVASCRIPT?



НЕ СОВСЕМ

- * Объекты в JS куда более могущественная структура, но **под капотом она использует как массивы, так и структуры**
- * Поговорим про это на отдельной лекции
- * В JS встроенных структур нет, но вы уже вероятно догадались, что **их можно добавить** используя [ArrayBuffer](#) и типизированные массивы

НО ЗАЧЕМ
НАМ ЭТО
НУЖНО?



НЕ ТОЛЬКО ЧИСЛА

- * Типизированные массивы из коробки **работают только с числами** – это не очень практично, ведь многие задачи требуют других типов
- * Описав схему структуры, мы сможем **закодировать её в типизированном массиве**
- * Структура может **содержать поля с другими структурами**, быть элементом массива...
- * Мы можем закодировать **структуру данных любой сложности**

С ТОЧКИ ЗРЕНИЯ ПРОИЗВОДИТЕЛЬНОСТИ

- * Объекты в JS работают всегда через ссылки, а в структурах мы **тонко контролируем этот момент**
- * Создание и удаление объектов **создают работу для сборщика мусора**
- * Если объектов много и живут они мало – это **огромная проблема**
- * Все **что мы делаем внутри `ArrayBuffer`** сборщик мусора не волнует
- * Мы можем **передать такую структуру без сериализации** в другой поток/процесс или по сети

КАКИЕ ОГРАНИЧЕНИЯ

- * В отличие от массива, структуру просто так **нельзя обойти поэлементно**, так как размер каждого поля может отличаться
- * Обход поддержать можно, но **это нужно программировать в рантайме**
- * Как и **нельзя добавлять новые элементы** – только то, что описано в схеме
- * Доступ к полю структуры, обычно, **осуществляется ассоциативно** – по ключу или имени свойства (которые описаны в схеме)
- * Но такая логика **требует поддержки на уровне языка или компилятора**

КОРТЕЖИ – ПОЧТИ СТРУКТУРЫ

- * Еще одна **фундаментальная структура данных**
- * Реализуются также, как и структуры,
но **вместо символический имен свойств используются индексы**
- * Кортежи можно сравнить с гетерогенным массивом,
где **тип каждого элемента описан в схеме**

НА ПРИМЕРЕ RUST

```
fn main() {  
    struct RGBA(u8, u8, u8, u8);  
  
    let black = RGBA(0, 0, 0, 1);  
  
    println!("Black is {} {} {} {}!", black.0, black.1, black.2, black.3);  
}
```

ТЕГИРОВАННЫЕ УКАЗАТЕЛИ

- * Массивы накладывают на нас архитектурное ограничение, что **размер элемента каждого массива должен быть заранее зафиксирован**
- * Получается, что мы **не можем хранить разные типы элементов в одном массиве**
- * Но JavaScript успешно **обходит это ограничение за счет использования указателей**
- * В добавок, в самом указателе «хранится признак», о том, что это за указатель
- * То есть **по значению отдельных бит** мы можем понять, что это за тип
- * Такие указатели называются **тегированные**

МАСШТАБИРУЕМ ПОДХОД

- * Можно создать структуру, в которой **отдельное поле хранит признак-типа структуры**
- * Такое поле **называют дискриминатор или дискриминант**
- * Тогда мы можем **выделить размер структуры под все нужные нам типы данных** и хранить признак, что за тип лежит в конкретно этой структуре
- * Как результат, мы сможем **хранить разные типы данных** в рамках такой структуры в одном массиве или поле другой структуры
- * Такая структура называется **перечислением или размеченным объединением**

РЕФЛЕКСИРУЕМ

- * В типизированных массивах можно хранить **не только числа или структуры/кортежи**
- * Но и **смешивать разные типы данных** с помощью перечислений
- * Разумеется, нам нужно **написать код кодирования дискриминатора**
- * Мы полностью подчинили себе типизированные массивы

second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

СТОЯНА, СТОЯНААА!

ПОСТАРАЙТЕСЬ ПОНЯТЬ СМЫСЛ

- * Структуры и кортежи отталкиваются от простой идеи – если у нас есть схема данных (какие поля и их тип), то **мы можем их закодировать**
- * Мы можем **статически посчитать отступ в памяти для каждого поля** имея адрес первого байта значения-структуры
- * Перечисления – это тоже структуры, которые позволяют **в рамках одного общего типа хранить разные типы**
- * Память в таком случае **выделяется под «максимальный» подтип**
- * Плюс нужно хранить дискриминатор, где хранится **тип конкретного значения перечисления**

ОК, КАК НАМ ЭТО
ВСЕ СДЕЛАТЬ?



```
const data = new ArrayBuffer(1024); // Размер указывается в байтах

const size = User.age.bytes + User.name.bytes;

function getUserAge(buffer, offset = 0) {
  return new Uint8Array(buffer, offset)[0];
}

function setUserAge(age, buffer, offset = 0) {
  new Uint8Array(buffer, offset)[0] = age;
}

function getUserName(buffer, offset = 0) {
  return new TextDecoder().decode(new Uint8Array(buffer, offset + User.age.bytes, User.name.bytes));
}

function setName(name, buffer, offset = 0) {
  new TextEncoder().encodeInto(name, new Uint8Array(buffer, offset + User.age.bytes, User.name.bytes));
}
```

```
const from = [{age: 42, name: "Bob"}, {age: 10, name: "Ben"}];

for (let i = 0; i < from.length; i++) {
  setUserAge(from[i].age, data, i * size);
  setName(from[i].name, data, i * size);
}

console.log(getUserAge(data, 0 * size));
console.log(getUserName(data, 0 * size));

console.log(getUserAge(data, 1 * size));
console.log(getUserName(data, 1 * size));
```

second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

СТОЯНА, СТОЯНААА!

ПОНИМАЮ ВАШИ ЧУВСТВА

- * Сейчас это все может выглядеть ужасно, но мы научимся с этим работать
- * Давайте разберем по шагам, что мы делаем в этом примере

СДВИГ И ДЛИНА

```
const buf = new ArrayBuffer(8);

// Использую память buf с 0 байта как массива Uint8Array длиной 4 (4 байта, так как Uint8Array)
const arr1 = new Uint8Array(buf, 0, 4);

// Использую память buf с 4 байта как массива Int32Array длиной 1 (4 байта, так как Int32Array)
const arr2 = new Int32Array(buf, 4, 1);

arr1[0] = 1;
arr1[1] = 2;
arr1[2] = 3;
arr1[3] = 4;

arr2[0] = -23413;

console.log(...arr1); // 1 2 3 4
console.log(...arr2); // -23413
```

РЕФЛЕКСИРУЕМ

- * С помощью сдвига мы **можем использовать разные части одной и той же памяти**
- * При этом длину массива задавать не обязательно,
но тогда может произойти «наложение» одного массива на другой

НАПРИМЕР

```
const buf = new ArrayBuffer(8);  
const arr1 = new Uint8Array(buf, 0);  
const arr2 = new Int32Array(buf, 4, 1);  
  
arr2[0] = -23413;  
  
arr1[0] = 1; arr1[1] = 2;  
arr1[2] = 3; arr1[3] = 4;  
arr1[4] = 1; arr1[5] = 2;  
arr1[6] = 3; arr1[7] = 4;  
  
console.log(...arr1); // 1 2 3 4 1 2 3 4  
console.log(...arr2); // Внезапно, 67305985
```

ПОЛЕЗНЫЙ API

```
const buf = new ArrayBuffer(128);  
const arr = new Uint16Array(buf, 4, 10);  
  
console.log(arr.length); // 10  
console.log(arr.byteLength); // 20 (2 байта на элемент * 10)  
console.log(arr.byteOffset); // 4  
  
console.log(arr.BYTES_PER_ELEMENT); // 2  
console.log(Uint16Array.BYTES_PER_ELEMENT); // 2
```

КОДИРОВАНИЕ СТРОК

- * Одну и ту же строку можно закодировать используя **разные кодовые таблицы и кодировки**
- * Строки в JS используют кодировку UTF-16, но это скорее легаси
- * Сейчас в мире стандарт UTF-8, поэтому лучше использовать его
- * В чем разница – разберем на отдельной лекции
- * А сейчас просто используем API `TextEncoder` и `TextDecoder` – они **позволяют кодировать и декодировать JS как UTF-8 буферы**

```
const data = new ArrayBuffer(1024); // Размер указывается в байтах

const size = User.age.bytes + User.name.bytes;

function getUserAge(buffer, offset = 0) {
  return new Uint8Array(buffer, offset)[0];
}

function setUserAge(age, buffer, offset = 0) {
  new Uint8Array(buffer, offset)[0] = age;
}

function getUserName(buffer, offset = 0) {
  return new TextDecoder().decode(new Uint8Array(buffer, offset + User.age.bytes, User.name.bytes));
}

function setName(name, buffer, offset = 0) {
  new TextEncoder().encodeInto(name, new Uint8Array(buffer, offset + User.age.bytes, User.name.bytes));
}
```

```
const from = [{age: 42, name: "Bob"}, {age: 10, name: "Ben"}];

for (let i = 0; i < from.length; i++) {
  setUserAge(from[i].age, data, i * size);
  setName(from[i].name, data, i * size);
}

console.log(getUserAge(data, 0 * size));
console.log(getUserName(data, 0 * size));

console.log(getUserAge(data, 1 * size));
console.log(getUserName(data, 1 * size));
```

ПАЗЛ НАЧИНАЕТ СХОДИТЬСЯ

- * Используя схему типа-структуры и сдвиги можно написать логику работы с памятью так, чтобы она работала согласно заданным контрактам
- * Типизированный массив над буфером ничего не копирует – только определяет контракты
- * Но тут нас ждет несколько нюансов

ВЫРАВНИВАНИЕ ДАННЫХ



```
const buf = new ArrayBuffer(8);
```

```
const arr = new Uint16Array(buf, 1); // RangeError: start offset of Uint16Array should be a multiple of 2
```

В ЧЕМ ПРОБЛЕМА

- * Размер одного элемента `Uint16Array` – это 2 байта
- * Если мы начнем писать в память со сдвигом в 1 байт, то может возникнуть ситуация, что **последний элемент массива поместиться в память лишь наполовину**
- * Чтобы избежать этого вводится простое правило:
адрес данных (индекс) должен быть кратен размеру данных
- * Для `Uint16Array` – должно быть кратно 2, для `Uint32Array` – 4
- * Это называется выравниванием

ЭТО АРХИТЕКТУРНОЕ ТРЕБОВАНИЕ

- * А не особенность типизированных массивов в JS
- * Адрес любого значения в памяти должен быть выравнен для эффективной работы
- * В языках с поддержкой структур за нас это делает компилятор/среда



```
const U8 = {  
  bytes: 1,  
  view: Uint8Array  
};  
  
const U16 = {  
  bytes: 1,  
  view: Uint16Array  
};  
  
// Размер с учетом выравнивания 4 байта  
const User = {  
  age: U8,  
  // Между age и lvl нужно пропустить 1 байт  
  lvl: U16  
};
```

ЭТО ЕЩЕ НЕ ВСЕ

- * Структуры и кортежи должны иметь выравнивание кратное размеру самого большого поля
- * Для перечисления – кратно размеру самого большого варианта
- * А для массива – кратно размеру элемента

second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

СТОЯНА, СТОЯНААА!

ЗАПОМНИТЕ ПРАВИЛО ВЫРАВНИВАНИЯ

- * Как отдельный атомарных типов (кратно размеру)
- * Так и составных: массивы, структуры, кортежи и перечисления (кратно размеру наибольшего члена)
- * **Выравнивание можно посчитать статически** на основе схемы данных

МОЖНО СОЗДАТЬ ОБЩУЮ БИБЛИОТЕКУ



```
import { Struct, Tuple, TypedArray, U8, U16, FixedAsciiString } from './struct.js'

export const Color = Tuple(U8, U8, U8);

export const Person = new Struct({
  age: U8,
  id: U16,
  firstName: FixedAsciiString(8),
  lastName: FixedAsciiString(8),
  color: Color
});

export const PersonArray = new TypedArray(Person, 1e6);
```

КАКИЕ ТУТ ЕЩЕ ПЛЮСЫ

- * Если ваши структуры над бинарными данными работают как View, то вам не только не нужно сериализовывать данные – но и **не нужно их парсить**
- * Парсинг гигабайта данных **будет занимать столько же**, сколько и килобайта
- * С тем же JSON чем больше файл – тем дольше парсинг

ОТКРЫТЫЙ УРОК

- ✱ К сегодняшней лекции **доступен дополнительный открытый урок**
- ✱ Там мы на практике **напишем возможный вариант такой библиотеки**
- ✱ Обязательно посмотрите его после лекции и поиграйтесь с [КОДОМ](#)

API ТИПИЗИРОВАННЫХ МАССИВОВ

- ✱ Очень сильно похож на API обычных массивов
- ✱ Но есть методы, которые нужно отдельно подсветить

КОПИРОВАНИЕ



```
const data = new Uint8Array([1, 2, 3, 4, 5]);
```

```
const slice = data.subarray(1, 4);
```

```
console.log(...slice); // 2 3 4
```

```
// В отличие от метода slice(), subarray не копирует память,  
// а просто создает подмассив с заданным сдвигом и длиной
```

```
console.log(slice.buffer === data.buffer); // true
```

```
// Set копирует другой массив в текущий,  
// вторым параметром можно задать сдвиг в текущем массиве  
data.set(new TextEncoder().encode("hello"));
```

ПАРСИНГ И СЕРИАЛИЗАЦИЯ В СТРОКУ (ES2025)

```
const data = new Uint8Array([29, 233, 101, 161]);

console.log(data.toBase64()); // "HelloQ=="
console.log(data.toHex()); // "1de965a1"

data.fill(0);
data.setFromBase64("HelloQ==");
console.log(data); // [29, 233, 101, 161]

data.fill(0);
data.setFromHex("1de965a1");
console.log(data); // [29, 233, 101, 161]

data.fill(0);

console.log(Uint8Array.fromBase64("HelloQ=="));
console.log(Uint8Array.fromHex("1de965a1"));
```

ЕСТЬ ЕЩЕ КОЕ-ЧТО

- ✱ Когда мы разбираем бинарные данные, то может быть **неудобно работать с ними как с массивом конкретного типа**
- ✱ В JS существует **специальный API** для таких задач

ВСТРЕЧАЙТЕ, **DataView**

```
const buffer = new ArrayBuffer(1024);
const view = new DataView(buffer);

// Записываем разные типы данных (первый аргумент задает сдвиг в байтах)
view.setInt8(0, 42);           // 1 байт: 42
view.setUint16(1, 65535);      // 2 байта: 65535 (максимальное 16-битное число)
view.setInt32(3, -1000);       // 4 байта: -1000
view.setFloat32(7, 3.14159);   // 4 байта: 3.14159
view.setFloat64(11, 2.71828);  // 8 байтов: 2.71828

// Читаем значения
console.log("Int8 (0):", view.getInt8(0));           // 42
console.log("Uint16 (1):", view.getUint16(1));        // 65535
console.log("Int32 (3):", view.getInt32(3));          // -1000
console.log("Float32 (7):", view.getFloat32(7));     // 3.14159
console.log("Float64 (11):", view.getFloat64(11));   // 2.71828
```

НЮАНСИКИ ПОДЪЕХАЛИ



```
const arr = new Uint32Array([567891]);

const view = new DataView(arr.buffer);

console.log(arr[0]);           // 567891
console.log(view.getUint32(0)); // Внезапно, 1403652096
console.log(view.getUint32(0, true)); // 567891
```

second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

СТОЯНА, СТОЯНААА!

КРЕПИТЕСЬ, МЫ ПОЧТИ ЗАКОНЧИЛИ

- * Дело в том, что разные процессоры по разному интерпретируют байты
- * x86 и ARM считают байты в порядке от младшего к старшему (Little-Endian)
- * А есть такие, кто делает это наоборот (Big-Endian)
- * Проблемы возникают, когда мы не знаем какой порядок использовался при кодировании данных, когда мы их декодируем
- * Типизированные массивы JS используют «родной» порядок системы
- * А **DataView** Big-Endian, но можно переключить в Little-Endian

НАПРИМЕР

```
const arr = new Uint32Array([567891]);

const view = new DataView(arr.buffer);

console.log(new Uint8Array(arr.buffer)); // Uint8Array(4) [ 83, 170, 8, 0 ]

console.log(view.getUint32(0, true)); // 567891, читаем как [ 83, 170, 8, 0 ]
console.log(view.getUint32(0));      // 1403652096, читаем как [ 0, 8, 170, 83 ]

// Сохранили исходное значение, но как Big-Endian
view.setUint32(0, 567891);

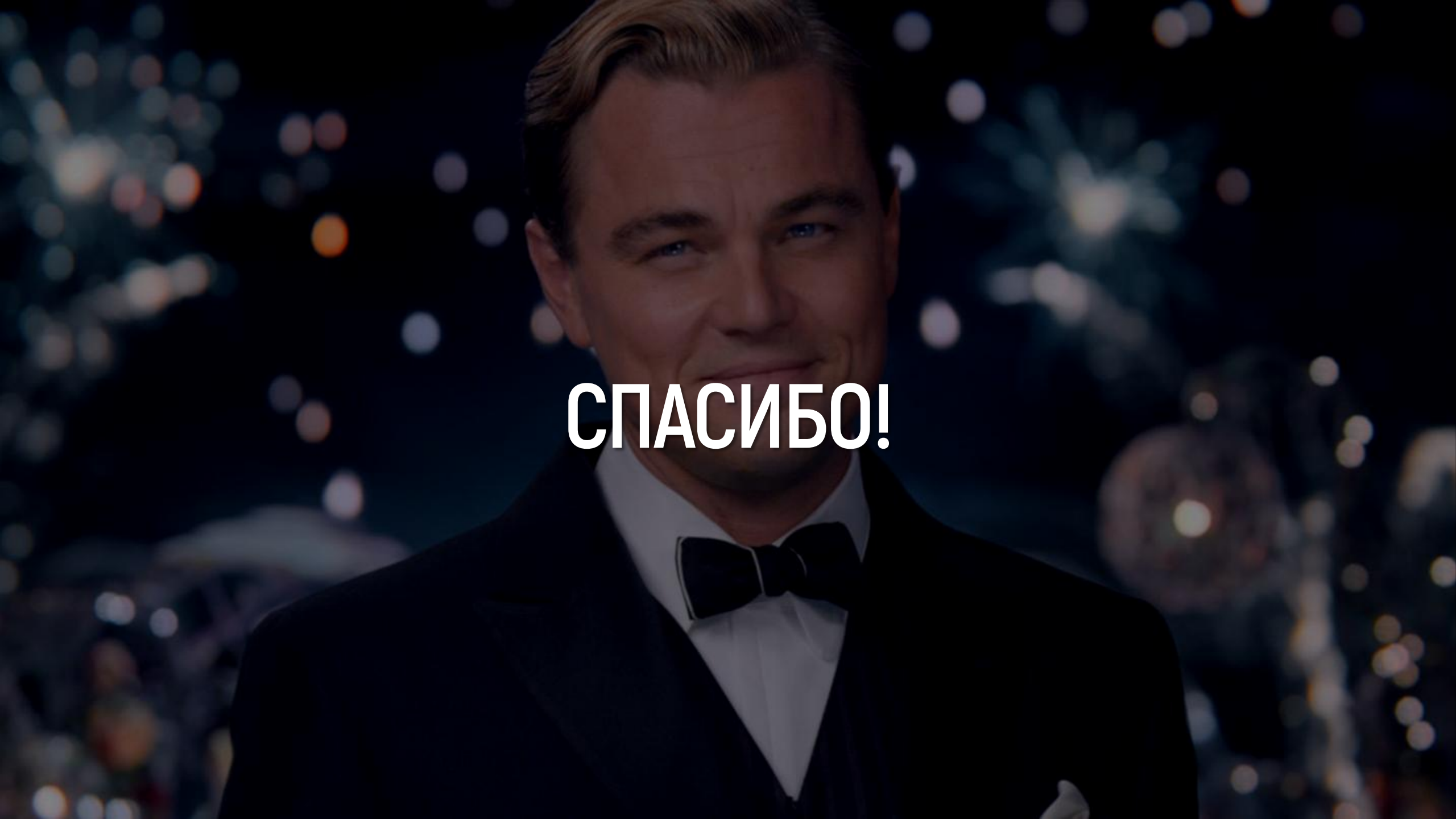
console.log(new Uint8Array(arr.buffer)); // Оп, Uint8Array(4) [ 0, 8, 170, 83 ]
```

КОГДА ЭТО ВАЖНО

- * Когда мы работаем с буфером полученным «из вне» (например, из сети)
- * И когда мы читаем значения **состоящие более чем из одного байта**
- * Но зачастую сейчас везде используются Little-Endian
- * Для строк в кодировке UTF-16 даже **есть специальный BOM-тег**, показывающей с каким порядком байт был закодирован текст
- * А вот для UTF-8 это уже не нужно

ПОДВЕДЕМ ИТОГИ

- * Типизированные массивы в JS – куда более близки к фундаментальному понятию массив в Computer Science, нежели обычные массивы JS
- * При этом работать с ними зачастую куда проще
- * А с другой стороны они открывают доселе невиданные для JS возможности
- * Появляется возможность работы с бинарными файлами и создание своих собственных низкоуровневых типов данных
- * Тонкое управление кэшем и сборкой мусора, бесплатная сериализация
- * Мы еще не раз вернемся к ним на нашем курсе

A close-up portrait of Leonardo DiCaprio, smiling slightly, wearing a dark tuxedo with a white shirt and a dark bow tie. The background is dark with out-of-focus bokeh lights in various colors (white, yellow, orange, blue), suggesting a night-time event or fireworks.

СПАСИБО!